WAMO Labs — Complete Interview Q&A: Cheatsheet

Muhammad Faizan Yousaf | MERN / Python / AI-ML | Fresh Graduate Role

SECTION 1 — Introduction & Behavioural

Q: Tell me about yourself.

➜ I'm a final-year CS student at FAST NUCES, graduating June 2026. I've built full-stack apps using MERN and Django, integrated AI features using RAG and NLP, and done two internships — one building a speech-to-text meeting pipeline, another on ML-based loan prediction. I apply cleanly for the MERN/AI role here.

■ *Keep it under 60 seconds. End with 'that's why I applied here' to close the loop.*

Q: Walk me through your background / stack.

➜ Frontend: React, Next.js. Backend: Node/Express, Django, FastAPI. DB: MongoDB, MySQL. AI: RAG, FAISS, NLP summarization. I've used all of these in real projects, not just tutorials.

Q: What is your strongest project?

➜ AI Knowledge OS — a full-stack app with semantic search using RAG and vector embeddings, JWT auth, and an AI chat interface. It solves real document retrieval problems, not a toy. Built end-to-end by me.

Q: Why WAMO Labs?

➜ Fast-paced engineering team, real-world products, and the role matches exactly what I've been building — MERN + AI integration. I want to work where my skills are used immediately, not shelved for 6 months.

Q: What is your weakness?

➜ I sometimes over-engineer solutions — adding more abstraction than needed. I've learned to ship simpler first and refactor only when complexity is actually justified.

■ *Never say 'I work too hard'. Pick a real one and show you're managing it.*

Q: Rate yourself in problem-solving / DB / communication (out of 5)?

➜ Problem-solving: 3.5 — solid fundamentals, still building speed on complex DSA. DB: 3.5 — strong on schema design and queries, learning query optimization. Communication: 4 — I explain clearly and ask the right questions.

■ *Don't say 5/5 for anything. They know you're fresh. Honest ratings with context = credible.*

Q: Where do you see yourself in 2 years?

➜ Contributing to production systems independently, mentoring juniors, and going deep in AI-integrated backend development. I want to be the person who bridges ML and software engineering on the team.

Q: Are you comfortable with onsite / night shift / 1PM onward?

➜ Yes on all counts. I applied for an onsite role in Lahore, I'm prepared for it. Flexible hours are fine — I prioritize output over fixed schedules.

SECTION 2 — OOP (JavaScript Focus)

Q: What are the 4 pillars of OOP?

➜ Encapsulation, Abstraction, Inheritance, Polymorphism. Each solves a different design problem.

Q: Explain Encapsulation.

➜ Bundling data and the methods that operate on it into one unit, and restricting direct access to internals.

```
class BankAccount {
#balance = 0; // private field
deposit(amt) { this.#balance += amt; }
getBalance() { return this.#balance; }
}
```

■ *The outside world calls getBalance() — it never touches #balance directly.*

Q: Explain Abstraction.

➜ Hiding complex implementation details and exposing only what's necessary. You use a function without knowing its internals.

```
// You call Array.sort() — you don't implement the sorting algorithm
// Abstract class pattern in JS:
class Shape { area() { throw new Error('Must implement area()'); } }
```

Q: Explain Inheritance.

➜ A child class inherits properties and methods from a parent class, enabling code reuse.

```
class Animal { speak() { return 'some sound'; } }
class Dog extends Animal { speak() { return super.speak() + ' + bark'; } }
new Dog().speak(); // 'some sound + bark'
```

Q: Explain Polymorphism.

➜ The same method name behaves differently depending on the object calling it.

```
class Cat extends Animal { speak() { return 'meow'; } }
class Dog extends Animal { speak() { return 'woof'; } }
const animals = [new Cat(), new Dog()];
animals.forEach(a => console.log(a.speak())); // meow, woof
```

Q: Composition vs Aggregation — what's the difference?

➜ Both are 'Has-A' relationships. Composition is STRONG ownership — the child cannot exist without the parent. Aggregation is WEAK — the child lives independently.

```
// COMPOSITION: Engine dies when Car dies
class Car { constructor() { this.engine = new Engine(); } }
// AGGREGATION: Player exists without Team
class Team { constructor(player) { this.player = player; } }
const p = new Player();
const t = new Team(p); // p still exists if t is destroyed
```

Q: How does a derived class call a base class method?

➜ Using super.methodName() inside the derived class.

```
class Vehicle { describe() { return 'I am a vehicle'; } }
class Car extends Vehicle {
describe() { return super.describe() + ' — specifically a car'; }
}
```

SECTION 3 — JavaScript Deep Dive

Q: var vs let vs const — key differences?

➜ Scope, hoisting behavior, and re-assignment rules differ across all three.

	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	Yes — TDZ	Yes — TDZ
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No (binding)
Use today?	Avoid	Yes	Prefer always

Q: What is Hoisting?

➜ JavaScript moves declarations to the top of their scope before execution. var is hoisted AND initialized to undefined. let/const are hoisted but NOT initialized — accessing them before the declaration line throws a ReferenceError.

```
console.log(a); // undefined (var is hoisted + initialized)
var a = 5;
console.log(b); // ReferenceError — Temporal Dead Zone
let b = 10;
```

Q: What is the Temporal Dead Zone (TDZ)?

➜ The period between when a let/const variable is hoisted and when it is actually initialized. Any access in this window throws ReferenceError.

■ *TDZ = the 'dead' gap between top of block and the declaration line.*

Q: What is a Promise? How does async/await work?

➜ A Promise is an object representing a future value — either resolved or rejected. async/await is syntax sugar over Promises — it makes async code look synchronous.

```
// Promise chain
fetch('/api').then(r => r.json()).then(data => console.log(data));
// async/await — same thing, cleaner
async function getData() {
const r = await fetch('/api');
const data = await r.json();
return data;
}
```

Q: Promise.all vs Promise.race?

➜ Promise.all waits for ALL to resolve — if one rejects, everything fails. Promise.race returns the result of whichever settles first (resolve or reject).

```
// All must succeed
const [u, p] = await Promise.all([fetchUser(), fetchPosts()]);
// Fastest wins (e.g. timeout pattern)
const result = await Promise.race([fetchData(), timeout(3000)]);
```

Q: Scenario: Call API1, then only if resolved call API2.

➜ Use sequential await — each call waits for the previous one.

```
async function run() {
const r1 = await fetch('/api1'); // waits here
if (!r1.ok) throw new Error('API1 failed');
const r2 = await fetch('/api2'); // only runs if r1 resolved
return await r2.json();
}
```

Q: What is the Event Loop?

➜ JS is single-threaded. The event loop picks tasks from the callback queue and executes them when the call stack is empty. Microtasks (Promises) run before macrotasks (setTimeout).

■ *Order: Call Stack → Microtask Queue (Promises) → Macrotask Queue (setTimeout, setInterval)*

Q: What is closure?

➜ A function that retains access to its outer scope even after the outer function has returned.

```
function counter() {
let count = 0;
return () => ++count; // inner fn closes over 'count'
}
const inc = counter();
inc(); // 1 inc(); // 2 — count persists
```

Q: == vs === ?

➜ == does type coercion before comparing. === checks both value AND type — no coercion.

```
0 == '0' // true (coercion)
0 === '0' // false (different types)
```

■ *Always use === in production code.*

SECTION 4 — DBMS: Normalization, ACID, Transactions, Indexing

Q: What is Normalization? When do you use it?

➜ Normalization eliminates data redundancy by organizing data into related tables. Use it in write-heavy systems (OLTP — banking, e-commerce) where you need data integrity.

Q: What is Denormalization? When do you use it?

➜ Intentionally adding redundancy to reduce JOINs and speed up reads. Use it in read-heavy systems — dashboards, analytics, reporting.

■ *Trade-off: faster reads, but updates must touch multiple places.*

Form	Rule — what it eliminates
1NF	Atomic columns only. No repeating groups or arrays in a cell.
2NF	1NF + No partial dependency. Every non-key column depends on the FULL primary key.
3NF	2NF + No transitive dependency. Non-key column must not depend on another non-key column.
BCNF	Stricter 3NF. Every determinant must be a candidate key.

Q: What are ACID properties? (CRITICAL)

➜ ACID ensures database transactions are reliable even in failure scenarios.

Property	Meaning	Real-world example
Atomicity	All or nothing. Either full transaction commits or it fully rolls back.	Bank transfer: debit Rs.500 AND credit Rs.500 — both happen or neither c
Consistency	DB moves from one valid state to another. All constraints must hold.	Account balance never goes negative after any transaction.
Isolation	Concurrent transactions do not interfere with each other.	Two users booking same seat — only one succeeds, other sees updated s
Durability	Once committed, data survives crashes. Stored to disk or write-ahead log.	Order confirmed — still there after server restarts.

Q: What is a Transaction? Show the syntax.

➜ A transaction is a unit of work that must complete fully or not at all. It groups multiple SQL statements into one atomic operation.

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT; -- saves both changes
-- If any error: ROLLBACK; -- undoes everything
```

Q: What are Isolation Levels?

➜ Control how much one transaction is exposed to changes from other concurrent transactions. Higher isolation = safer but slower.

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
Read Uncommitted	YES	YES	YES	Fastest
Read Committed	No	YES	YES	Fast
Repeatable Read	No	No	YES	Medium
Serializable	No	No	No	Slowest

Q: Dirty Read vs Non-Repeatable Read vs Phantom Read?

➜ Dirty Read: reading uncommitted data from another transaction (may be rolled back). Non-Repeatable Read: same row gives different value if re-read. Phantom Read: a re-query returns new rows that didn't exist before.

Q: What is Indexing? When should you use it?

➜ An index is a data structure (usually B-Tree) that speeds up SELECT queries on indexed columns. The trade-off: reads get faster, writes get slightly slower (index must update).

```
CREATE INDEX idx_user_email ON users(email);
-- Now: SELECT * FROM users WHERE email = 'x@y.com' is fast
```

■ *Use on: WHERE columns, JOIN keys, ORDER BY columns. Avoid on: rarely-queried columns, small tables.*

Q: Multiplicity — 1:1, 1:M, M:N?

➜ 1:1 — one person, one passport. 1:M — one user, many orders. M:N — many students enroll in many courses → needs a junction table (enrollment) with both foreign keys.

Q: InDrive schema: design ERD and write a query.

➜ Tables: Rider, Driver, Booking (junction). Rider 1:M Booking. Driver 1:M Booking. Rider and Driver are M:N via Booking.

```
-- Tables
Rider(rider_id PK, name, phone, rating)
Driver(driver_id PK, name, phone, vehicle, rating, is_available)
Booking(booking_id PK, rider_id FK, driver_id FK,
pickup, dropoff, status, price, created_at)
-- Query: all completed rides for rider 101
SELECT b.booking_id, d.name AS driver, b.pickup, b.dropoff, b.price
FROM Booking b
JOIN Driver d ON b.driver_id = d.driver_id
WHERE b.rider_id = 101 AND b.status = 'completed'
ORDER BY b.created_at DESC;
```

SECTION 5 — DSA: Reverse Number, Stack, Linked List

Q: Reverse a number.

➜ Extract digits one by one using modulo, build the reversed number by multiplying existing result by 10 each step.

```
function reverseNum(n) {
let rev = 0, num = Math.abs(n);
while (num > 0) {
rev = rev * 10 + (num % 10);
num = Math.floor(num / 10);
}
return n < 0 ? -rev : rev;
}
reverseNum(1234); // 4321
reverseNum(-56); // -65
```

Q: Valid Parenthesis — Stack problem.

➜ Push opening brackets onto stack. For each closing bracket, pop and check if it matches. If stack is empty at end — valid.

```
function isValid(s) {
const stack = [];
const map = { ')':'(', ']':'[', '}':'{' };
for (let c of s) {
if ('([{'.includes(c)) stack.push(c);
else if (stack.pop() !== map[c]) return false;
}
return stack.length === 0;
}
isValid('()[]{}'); // true
isValid('([)]'); // false
```

Q: Linked list of linked lists — sort all values (O(n²) ok).

➜ Flatten all inner list values into an array, sort it, optionally rebuild as a linked list.

```
// Node structure: { val, next, innerList }
function flattenAndSort(head) {
const vals = [];
let curr = head;
while (curr) {
let inner = curr.innerList;
while (inner) {
vals.push(inner.val);
inner = inner.next;
}
curr = curr.next;
}
return vals.sort((a, b) => a - b); // O(n log n) sort
}
// Rebuild as linked list from sorted array:
function buildList(arr) {
let dummy = {}, curr = dummy;
for (let v of arr) { curr.next = { val: v, next: null }; curr = curr.next; }
return dummy.next;
}
```

Q: What is time complexity of common operations?

➜ Know these by heart — they test this casually.

Operation	Best Case	Average	Worst Case
Array access	O(1)	O(1)	O(1)
Linear search	O(1)	O(n)	O(n)
Binary search	O(1)	O(log n)	O(log n)
Bubble sort	O(n)	O(n²)	O(n²)
Merge sort	O(n log n)	O(n log n)	O(n log n)
Hash map get/set	O(1)	O(1)	O(n) — collision
Stack push/pop	O(1)	O(1)	O(1)

SECTION 6 — MERN Stack Concepts

Q: What does MERN stand for?

➜ MongoDB (document DB), Express.js (Node web framework), React (frontend UI library), Node.js (JS runtime). Full JavaScript stack — same language frontend to backend.

Q: How does a REST API work?

➜ Client sends HTTP request (GET/POST/PUT/DELETE) to a URL endpoint. Server processes it, queries DB if needed, and returns JSON response. Express handles routing.

```
// Express route example
app.get('/users/:id', async (req, res) => {
const user = await User.findById(req.params.id);
res.json(user);
});
```

Q: What is middleware in Express?

➜ Functions that run between receiving a request and sending a response. Used for auth, logging, error handling, body parsing.

```
app.use((req, res, next) => {
console.log(req.method, req.url); // logging middleware
next(); // pass to next middleware or route
});
```

Q: What is JWT? How does auth work?

➜ JSON Web Token. Server issues a signed token on login. Client stores it and sends it in the Authorization header on every request. Server verifies the signature — no DB lookup needed.

```
// Token structure: header.payload.signature (base64)
// Verify incoming request:
const token = req.headers.authorization?.split(' ')[1];
const decoded = jwt.verify(token, process.env.SECRET);
```

Q: SQL vs NoSQL — when to choose which?

➜ SQL: structured data, complex relationships, ACID needed (banking, ERP). NoSQL: flexible schema, high scale, hierarchical or document data (user profiles, logs, real-time apps).

	SQL (PostgreSQL, MySQL)	NoSQL (MongoDB)
Schema	Fixed, rigid	Flexible, dynamic
Relationships	JOINs, foreign keys	Embedded docs or references
Scaling	Vertical (scale up)	Horizontal (scale out)
ACID	Full support	Limited (depends on version)
Best for	Finance, ERP, structured data	Catalogs, logs, real-time, JSON-heavy

Q: What is React's virtual DOM?

➜ React keeps a lightweight copy of the real DOM in memory. On state change it diffs old vs new virtual DOM (reconciliation) and updates ONLY the changed real DOM nodes — much faster than re-rendering everything.

Q: useState vs useEffect?

➜ useState stores local component state — any change triggers re-render. useEffect runs side effects (API calls, subscriptions, timers) after render. Dependency array controls when it re-runs.

```
const [data, setData] = useState(null);
useEffect(() => {
fetch('/api').then(r=>r.json()).then(setData);
}, []); // [] = run once on mount only
```

SECTION 7 — End-of-Interview Questions + Final Tips

Questions to ask THEM at the end:

• What does the first 30 days look like for someone joining this team?

• What's the biggest technical challenge the team is facing right now?

• How does code review work here — what does 'done' mean on this team?

• What tech stack decisions are being made that a new hire would have input on?

Behavior under technical pressure:

• If you don't know — say: 'I haven't implemented that specifically, but here's how I'd approach it...'

• Think out loud always — silence is worse than a wrong guess

• For DSA: state your approach first, then code — don't code blindly

• If they push back on your answer — don't collapse. Defend your reasoning, then ask what they had in mind.

Topic	The Line They Want to Hear
ACID crash scenario	'Atomicity ensures rollback; Durability ensures WAL persists data to disk before confirming commit.'
Why not use var?	'Function scope + silent hoisting to undefined causes unpredictable bugs. let/const give block scope and fail loudly.'
Normalization vs Speed	'Normalize for integrity in write-heavy systems; denormalize strategically for read speed when joins become a bottler
Promise.all failure	'One rejection short-circuits the whole batch. Use Promise.allSettled if you need all results regardless of failures.'
When to index	'Index when a column appears frequently in WHERE, JOIN, or ORDER BY. Skip it on low-cardinality or rarely-querie
Composition in 1 line	'Composition is ownership — the child cannot outlive the parent. Aggregation is association — child is independent.'

Prepare your intro, know your projects cold, and think out loud. That's 80% of the battle.